

**EXERCISE 12.11**

Try to write `compose` on `Monad`. It's not possible, but it is instructive to attempt it and understand why this is the case.

```
def compose[G[_]](G: Monad[G]): Monad[({type f[x] = F[G[x]]})#f]
```

12.6 Traversable functors

We discovered applicative functors by noticing that our `traverse` and `sequence` functions (and several other operations) didn't depend directly on `flatMap`. We can spot another abstraction by generalizing `traverse` and `sequence` once again. Look again at the signatures of `traverse` and `sequence`:

```
def traverse[F[_],A,B](as: List[A])(f: A => F[B]): F[List[B]]
def sequence[F[_],A](fas: List[F[A]]): F[List[A]]
```

Any time you see a concrete type constructor like `List` showing up in an abstract interface like `Applicative`, you may want to ask the question, "What happens if I abstract over this type constructor?" Recall from chapter 10 that a number of data types other than `List` are `Foldable`. Are there data types other than `List` that are *traversable*? Of course!

**EXERCISE 12.12**

On the `Applicative` trait, implement `sequence` over a `Map` rather than a `List`:

```
def sequenceMap[K,V](ofa: Map[K,F[V]]): F[Map[K,V]]
```

But traversable data types are too numerous for us to write specialized `sequence` and `traverse` methods for each of them. What we need is a new interface. We'll call it `Traverse`:⁶

```
trait Traverse[F[_]] {
  def traverse[G[_]:Applicative,A,B](fa: F[A])(f: A => G[B]): G[F[B]] =
    sequence(map(fa)(f))
  def sequence[G[_]:Applicative,A](fga: F[G[A]]): G[F[A]] =
    traverse(fga)(ga => ga)
}
```

⁶ The name `Traversable` is already taken by an unrelated trait in the Scala standard library.